

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом

Студентка гр. 4345

Савонькина Е.Н.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить принцип работы алгоритма поиска с возвратом. Решить с его помощью задачу.

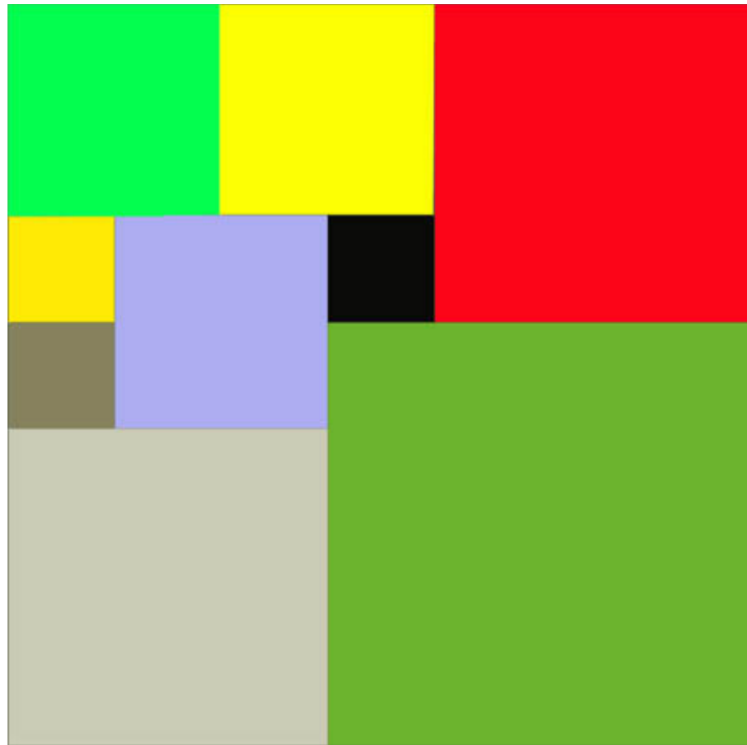
Основные теоретические положения.

Поиск с возвратом (backtracking) — это универсальный метод для решения задач, где необходимо перебрать все возможные варианты из заданного множества. Решение строится постепенно: на каждом шаге формируется частичное решение путем добавления нового элемента. Если дальнейшее расширение частичного решения невозможно, алгоритм откатывается назад к предыдущему состоянию и пробует другой вариант. Так можно найти все возможные решения задачи.

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N-1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу — квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N ($2 \leq N \leq 20$).

Выходные данные

Одно число K , задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу (квадрат) заданного размера N . Далее должны идти K строк, каждая из которых должна содержать три целых числа x , y и w , задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант 2р.

Рекурсивный бэктрекинг. Исследование времени выполнения от размера квадрата.

Основная идея рекурсивного алгоритма.

Алгоритм использует метод поиска с возвратом (backtracking) для разбиения квадрата размера $SIZE \times SIZE$ на наименьшее количество меньших квадратов.

Основные шаги:

1. Поиск свободной клетки: выбирается самая верхняя и левая незаполненная клетка на поле. Если свободных клеток нет, значит, текущее размещение квадратов завершено, и алгоритм фиксирует найденное решение.
2. Определение максимального квадрата: для найденной свободной клетки определяется наибольший возможный размер квадрата, который можно поставить, не выходя за пределы поля и не перекрывая уже установленные квадраты.
3. Проба квадратов всех размеров: начиная с максимального размера, алгоритм пытается поставить квадрат. Если квадрат помещается, он устанавливается на поле, и рекурсивно запускается дальнейший поиск для следующей свободной клетки.
4. Возврат: после завершения рекурсивного поиска или если квадрат больше не подходит, алгоритм удаляет последний установленный квадрат и пробует следующий размер.

5. Запись оптимального решения: если текущее размещение квадратов использовало меньше квадратов, чем предыдущие решения, оно сохраняется как лучшее.

6. Завершение: алгоритм исследует все возможные варианты расстановки квадратов и возвращает минимальное количество квадратов и их позиции на поле.

Оптимизации кода.

Первая оптимизация заключается в масштабировании размера исходной задачи. Перед началом работы алгоритма находится наибольший делитель заданного размера квадрата. После этого размер поля делится на этот делитель, и задача решается для уменьшенного квадрата. После нахождения решения координаты и размеры квадратов масштабируются обратно к исходному размеру. Это позволяет сократить размер поля и уменьшить глубину рекурсии.

Вторая оптимизация — отсечение неэффективных ветвей поиска. Во время работы алгоритма хранится лучшее найденное решение — минимальное количество квадратов. Если в текущей ветви поиска количество уже размещённых квадратов становится больше или равно лучшему найденному значению, дальнейший поиск по этой ветви прекращается. Это позволяет не рассматривать варианты, которые заведомо не приведут к более оптимальному решению.

Третья оптимизация связана с выбором свободной клетки. Алгоритм всегда выбирает самую левую и верхнюю свободную клетку на поле. Такой способ выбора позволяет избежать рассмотрения симметричных вариантов размещения квадратов и делает процесс заполнения более упорядоченным.

Четвёртая оптимизация заключается в ограничении максимального размера квадрата, который можно поставить в выбранную клетку. Размер квадрата определяется так, чтобы он не выходил за границы поля и не превышал половину размера исходного квадрата. Это уменьшает количество проверяемых вариантов.

Пятая оптимизация заключается в порядке перебора размеров квадратов. Сначала алгоритм пытается поставить квадрат максимально возможного размера, а затем постепенно уменьшает его. Это позволяет быстрее заполнить поле крупными квадратами и раньше получить решение с небольшим количеством квадратов.

Шестая оптимизация применяется для квадратов нечётного размера. В этом случае перед запуском рекурсивного поиска на поле сразу размещаются три квадрата: один большего размера и два немного меньшего. Такая начальная конфигурация уменьшает область поиска и позволяет ускорить работу алгоритма.

Описание функций.

1. *divisor(n)* – функция поиска делителя. Она находит наибольший делитель числа n , меньший самого числа. Перебор выполняется от $n // 2$ до 1. Если находится число, на которое n делится без остатка, функция возвращает его. Если таких делителей нет, возвращается 1. Результат используется для уменьшения размера исходной задачи.

2. *search_free_cell()* – функция поиска свободной клетки. Она последовательно просматривает игровое поле, начиная с верхней строки и двигаясь слева направо. Если находится клетка со значением 0, она считается свободной, и функция возвращает её координаты (x, y) . Если свободных клеток нет, возвращается $(None, None)$, что означает, что поле полностью заполнено.

3. *can_place(x, y, n)* – функция проверки возможности размещения квадрата. Она проверяет, можно ли разместить квадрат размера $n*n$ с левым верхним углом в клетке (x, y) . Сначала проверяется, не выходит ли квадрат за границы поля. Затем проверяется, свободны ли все клетки в области предполагаемого размещения. Если размещение возможно, функция возвращает `True`, иначе — `False`.

4. *place(x, y, n)* – функция установки квадрата. Она заполняет клетки игрового поля в области квадрата размера $n*n$, начиная с координат (x, y) ,

значением n , тем самым отмечая, что данные клетки заняты установленным квадратом.

5. $delete(x, y, n)$ – функция удаления квадрата. Она очищает клетки поля, которые занимал ранее установленный квадрат размера $n \times n$, устанавливая значение 0. Это используется при возврате алгоритма на предыдущий шаг поиска.

6. $max_square(x, y)$ – функция определения максимального размера квадрата. Она вычисляет наибольший возможный размер квадрата, который можно разместить в клетке (x, y) без выхода за границы поля и с учётом ограничения на максимальный размер квадрата. Функция возвращает найденный максимальный размер.

Основная рекурсивная функция `recursion()` .

Функция реализует основной алгоритм поиска решения методом рекурсивного бэктрекинга. Её задача — последовательно размещать квадраты на поле, перебирая возможные варианты их расположения и размеров. В процессе работы функция строит частичное решение, а при невозможности дальнейшего расширения выполняет возврат к предыдущему состоянию и пробует другие варианты. В результате находится разбиение квадрата на минимальное количество меньших квадратов.

Аргументы:

Функция не принимает аргументов. Для работы она использует глобальные переменные:

`field` — двумерный массив, представляющий текущее состояние поля;

`best_cnt` — минимальное количество квадратов, найденное на данный момент;

`best_pos` — список координат и размеров квадратов лучшего решения;

`tmp_cnt` — текущее количество размещённых квадратов;

`tmp_pos` — список координат и размеров квадратов текущего частичного решения;

`SIZE` — размер рассматриваемого поля.

Возвращаемое значение:

Функция не возвращает значение. Результат работы сохраняется в глобальных переменных `best_cnt` и `best_pos`, где хранится лучшее найденное разбиение квадрата.

Способ хранения частичных решений.

В процессе работы алгоритма частичные решения хранятся с использованием нескольких структур данных.

Текущее состояние заполнения поля хранится в двумерном массиве `field` размером `SIZE * SIZE`. Каждая ячейка массива содержит значение 0, если клетка свободна, или число, равное размеру квадрата, который занимает данную клетку. Таким образом, массив отражает текущее размещение квадратов на поле.

Для хранения текущего частичного решения используется список `tmp_pos`. В нём сохраняются кортежи вида (x, y, n) , где x и y — координаты левого верхнего угла квадрата, а n — размер его стороны. Каждый раз при установке нового квадрата информация о нём добавляется в этот список.

Количество квадратов в текущем частичном решении хранится в переменной `tmp_cnt`. Эта переменная увеличивается при добавлении квадрата и уменьшается при его удалении во время возврата алгоритма.

Лучшее найденное решение хранится в списке `best_pos`, который содержит координаты и размеры квадратов оптимального разбиения. Количество квадратов в этом решении хранится в переменной `best_cnt`.

При каждом завершении заполнения поля текущее частичное решение сравнивается с лучшим найденным. Если текущее решение использует меньше квадратов, оно копируется из `tmp_pos` в `best_pos`, а значение `tmp_cnt` сохраняется в `best_cnt`. Таким образом, в процессе работы алгоритма постоянно поддерживается информация о текущем и лучшем найденном разбиении.

Тестирование.

Таблица тестирования для корректных значений:

Размер столешницы n	Результат	Комментарии
2	4 1 1 1 2 1 1 1 2 1 2 2 1	Результат корректен.
5	8 1 1 3 4 1 2 1 4 2 4 3 2 3 4 1 3 5 1 4 5 1 5 5 1	Результат корректен.
10	4 1 1 5 6 1 5 1 6 5 6 6 5	Результат корректен.
11	11 1 1 6 7 1 5 1 7 5 7 6 3 10 6 2 6 7 1 6 8 1 10 8 1 11 8 1 6 9 3 9 9 3	Результат корректен.
19	13 1 1 10 11 1 9 1 11 9 11 10 3 14 10 6 10 11 1 10 12 1 10 13 4 14 16 1 15 16 1 16 16 4	Результат корректен.

	10 17 3	
	13 17 3	

Исследование.

График зависимости:



Из графика можно выделить следующее:

Время выполнения растёт вместе с возрастанием простых чисел. Это обусловлено оптимизацией, которая сводит решение большого квадрата к решению маленького.

Вывод.

В ходе выполнения работы была разработана программа, реализующая метод поиска с возвратом для разбиения квадрата на минимальное количество меньших квадратов. Применённые оптимизации позволили сократить перебор.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

```
def divisor(n):
    """Поиск наибольшего делителя"""
    for i in range(n // 2, 0, -1):
        if n % i == 0:
            return i
    return 1

def search_free_cell(field, SIZE):
    """Поиск свободной клетке (самой левой, самой верхней)"""
    for y in range(SIZE):
        for x in range(SIZE):
            if field[y][x] == 0:
                return x, y
    return None, None

def can_place(field, x, y, n, SIZE):
    """Проверка на заполненность"""
    if x + n > SIZE or y + n > SIZE:
        return False
    for i in range(y, y + n):
        for j in range(x, x + n):
            if field[i][j] != 0:
                return False
    return True

def place(field, x, y, n):
    """Установка квадрата"""
    for i in range(y, y + n):
        for j in range(x, x + n):
            field[i][j] = n

def delete(field, x, y, n):
    """Удаление квадрата"""
    for i in range(y, y + n):
        for j in range(x, x + n):
            field[i][j] = 0

def max_square(x, y, SIZE):
    """Выбор максимального размера"""
    return min(SIZE - x, SIZE - y, (SIZE + 1) // 2)
```

```

def recursion(field, SIZE, tmp_pos, tmp_cnt, best_cnt, best_pos):
    if tmp_cnt >= best_cnt[0]:
        print(f"Текущее количество квадратов {tmp_cnt} уже больше (или равно)
одного из прошлых, останавливаемся\n")
        return

    x, y = search_free_cell(field, SIZE)
    if x is None:
        best_cnt[0] = tmp_cnt
        best_pos.clear()
        best_pos.extend(tmp_pos)
        print(f"Квадрат полностью заполнен, количество: {best_cnt[0]}\n")
        return
    print(f"Текущее место: {x, y}")

    max_size = max_square(x, y, SIZE)
    print(f"Максимальный размер квадрата на этой клетке: {max_size}")

    for n in range(max_size, 0, -1):
        if can_place(field, x, y, n, SIZE):
            print(f"Пробуем квадрат {n}")
            print("Место есть, ставим")
            place(field, x, y, n)
            tmp_pos.append((x + 1, y + 1, n))
            tmp_cnt += 1

            for i in field:
                print(" ".join(map(str, i)))
            print("\nЗапускаем рекурсию")

            recursion(field, SIZE, tmp_pos, tmp_cnt, best_cnt, best_pos)

            print(f"Удаляем последний квадрат {n} на месте {x + 1, y + 1}")
            tmp_cnt -= 1
            tmp_pos.pop()
            delete(field, x, y, n)

    print("Поле после удаления:")
    for i in field:
        print(" ".join(map(str, i)))

```

```

print("Квадраты всех возможных размеров использованы")

def run_algorithm(n):
    gsd = divisor(n)
    SIZE = n // gsd

    field = [[0] * SIZE for _ in range(SIZE)]
    tmp_pos = []
    tmp_cnt = 0
    best_cnt = [float('inf')]
    best_pos = []

    print(f"Заданный размер {SIZE * gsd}, будем решать задачу для {SIZE}\n")

    if SIZE % 2 == 1:
        big_sq = (SIZE + 1) // 2
        small_sq = big_sq - 1

        print(f"Ставим 3 квадрата: один квадрат размера {big_sq} и два размера {small_sq}\n")

        place(field, 0, 0, big_sq)
        tmp_pos.append((1, 1, big_sq))

        place(field, big_sq, 0, small_sq)
        tmp_pos.append((big_sq + 1, 1, small_sq))

        place(field, 0, big_sq, small_sq)
        tmp_pos.append((1, big_sq + 1, small_sq))

        tmp_cnt = 3

        print("Расстановка начальных квадратов (для оптимизации):")
        for i in field:
            print(" ".join(map(str, i)))
        print()

    print("Запускаем рекурсию")
    recursion(field, SIZE, tmp_pos, tmp_cnt, best_cnt, best_pos)
    print("Рекурсия окончена")

    print(best_cnt[0])

```

```
for x, y, w in best_pos:
    print((x - 1) * gsd + 1, (y - 1) * gsd + 1, w * gsd)

if name == 'main':
    run_algorithm(int(input()))
```